Z3-alpha2@SMT-COMP 2026

John Zhengyang Lu[1] , **Avik Kumar**[2] , **Piyush Jha**[2] ,
Arie Gurfinkel[1] and **Vijay Ganesh**[2]

[1]University of Waterloo
[2]Georgia Institute of Technology
john.lu2@uwaterloo.ca

Abstract

This note presents `Z3-alpha2`, our SMT-COMP 2026 submission. Built on `Z3-4.16.0`, `Z3-alpha2` optimizes the default `Z3` strategies for SMT-LIB benchmarks. `Z3-alpha2` adopts a two-stage approach. First, it uses MCTS guided by an LLM to search for a portfolio of high-performing and complementary strategies. Second, it employs `SMT-Select-Lite`, a machine learning-based algorithm selector, to map individual benchmarks to specific strategies from the portfolio.

1 Introduction

Modern SMT solvers, such as `Z3` [de Moura and Bjørner, 2008], allow users to customize strategies for specific use cases [de Moura and Passmore, 2013]. However, handcrafting an effective customized strategy requires a deep understanding of solver internals and substantial trial-and-error effort.

`Z3-alpha` [Lu *et al.*, 2024; Lu *et al.*, 2025] addresses this challenge with a Monte Carlo Tree Search (MCTS)-based framework that automatically synthesizes SMT strategies tailored to targeted domains. In our SMT-COMP 2026 submission, we extend `Z3-alpha` with Large Language Model (LLM) guidance, resulting in `Z3-alpha2`, where LLMs help steer the MCTS exploration process toward more promising strategies and improve search efficiency.

`Z3-alpha2` is still under active development, and the SMT-COMP 2026 submission represents a preliminary version. It is submitted as a derived solver based on `Z3`-4.16.0, participating in the divisions of *Arith*, *QF_Bitvec*, *QF_Datatypes*, *QF_LinearIntArith*, *QF_Equality+NonLinearArith*, *QF_NonLinearIntArith*, *QF_NonLinearRealArith* in the Single Query track.

2 Methodology

`Z3-alpha` adopts a two-stage approach: (1) searching for a portfolio of effective and complementary linear strategies, each consisting of a sequence of tactics; and (2) synthesizing these linear strategies into a single branched strategy that combines their strengths.

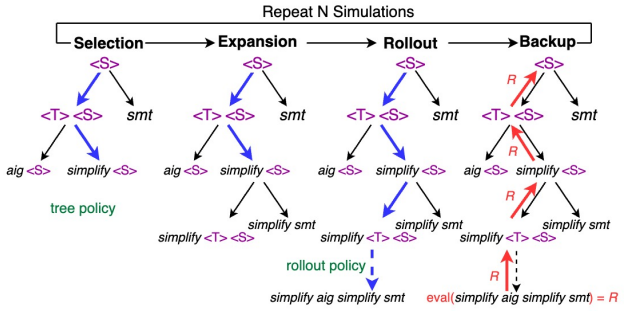


Figure 1: MCTS framework in Stage 1 of `Z3-alpha`

In `Z3-alpha2`, we enhance both stages. In Stage 1, the MCTS search is further guided by an LLM that provides background knowledge to improve the search process. In Stage 2, we incorporate machine learning-based algorithm selection to enable more fine-grained per-instance strategy selection.

2.1 LLM-guided MCTS

`Z3-alpha` models the process of synthesizing a linear strategy as a Markov Decision Process (MDP), where tactics are selected sequentially in a step-by-step manner. It then applies MCTS to explore the strategy space through many simulations, as illustrated in Figure 1. Each simulation consists of four phases: *selection*, which traverses the current search tree according to the tree policy; *expansion*, which adds new unexplored nodes to the tree; *rollout*, which completes a partial strategy and evaluates the resulting candidate strategy; and *backup*, which propagates the evaluation results back through the visited nodes. In `Z3-alpha2`, we augment both the selection and rollout phases with *LLM guidance*.

In the selection phase, unlike `Z3-alpha`, which adopts the classical UCT tree policy [Kocsis and Szepesvári, 2006], `Z3-alpha2` employs the PUCT policy introduced in AlphaGo [Silver *et al.*, 2016] and AlphaGo Zero [Silver *et al.*, 2017]. For a search state s and candidate action a, the PUCT score is defined as

$$\mathrm{PUCT}(s,a) = Q(s,a) + c\,P(s,a)\frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}.$$

Here, $Q(s,a)$ denotes the estimated value of selecting ac-

tion a in state s, computed as the average reward obtained from previous simulations passing through (s, a). $P(s, a)$ is the prior probability assigned to action a, $N(s, a)$ is the visit count of action a, $\sum_b N(s, b)$ is the total visit count of state s, and c is a constant controlling the exploration–exploitation trade-off.

The key advantage of PUCT over UCT is its ability to incorporate prior knowledge into the search process. While the value term $Q(s, a)$ reflects information gathered from previous simulations, the prior term $P(s, a)$ guides exploration toward actions that are expected to be promising even before substantial search effort has been invested in them. This often leads to more efficient exploration of the search space, particularly in domains such as SMT strategy synthesis, where evaluating a candidate strategy through simulation is computationally expensive.

In AlphaGo Zero, the prior probability $P(s, a)$ is produced by a policy network trained through self-play. In contrast, `Z3-alpha2` derives $P(s, a)$ from an LLM. Given the current search state, the LLM estimates the usefulness of each candidate tactic, which estimates the usefulness of candidate tactics based on the current search state and its knowledge of the base `Z3` solver and SMT solving.

In addition to guiding selection, the LLM is also used during rollouts. Rather than generating the remainder of a strategy by randomly selecting tactics, `Z3-alpha2` prompts the LLM to complete the partial strategy, leveraging its domain knowledge to produce higher-quality rollout trajectories.

Currently, the implementation of the LLM guidance is integrated into the coding-agent workflow such as Claude Code and Cursor, enabling access to a variety of LLMs throughout the whole competition preparation. We observe that the LLM guidance helps prune unpromising strategies early, allowing the search to focus on more promising regions of the strategy space.

2.2 Learning-based Algorithm Selection

After a complementary portfolio of linear strategies is identified in Stage 1,`Z3-alpha` uses an MCTS-based approach to combine them into a single branched strategy that maps individual formulas to suitable linear strategies. However, the branching decisions depend on only a small set of formula predicates, and the branching depth is restricted to at most three levels, resulting in low granularity in per-instance strategy selection.

To address this limitation, `Z3-alpha2` replaces the MCTS-based branching mechanism with `SMT-Select-Lite` [Lu *et al.*, 2026], a newly introduced machine learning-based algorithm selection framework for SMT problems. For each participating logic, `Z3-alpha2` trains a dedicated `SMT-Select-Lite` model that maps individual benchmarks to the most suitable strategy identified during the MCTS stage.

Acknowledgments

We acknowledge the authors of `Z3` for their foundational work, which our solver is built upon.

References

[de Moura and Bjørner, 2008] Leonardo de Moura and Nikolaj Bjørner. Z3: an efficient SMT solver. In *2008 Tools and Algorithms for Construction and Analysis of Systems*, pages 337–340. Springer, Berlin, Heidelberg, March 2008.

[de Moura and Passmore, 2013] Leonardo de Moura and Grant Olney Passmore. The strategy challenge in SMT solving. *Automated Reasoning and Mathematics: Essays in Memory of William W. McCune*, pages 15–44, 2013.

[Kocsis and Szepesvári, 2006] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.

[Lu *et al.*, 2024] Zhengyang Lu, Stefan Siemer, Piyush Jha, Joel Day, Florin Manea, and Vijay Ganesh. Layered and staged Monte Carlo Tree Search for SMT strategy synthesis. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 1907–1915, 2024.

[Lu *et al.*, 2025] Zhengyang John Lu, Joel Day, Piyush Jha, Paul Sarnighausen-Cahn, Stefan Siemer, Florin Manea, and Vijay Ganesh. Novel tree-search method for synthesizing SMT strategies. *Acta Informatica*, 62(3):28, 2025.

[Lu *et al.*, 2026] Zhengyang Lu, Paul Sarnighausen-Cahn, Jiahao Chen, Arie Gurfinkel, Florin Manea, and Vijay Ganesh. Learning unified graph and language representations for SMT algorithm selection. In *The 32nd International Conference on Principles and Practice of Constraint Programming (CP 2026)*, 2026. To appear.

[Silver *et al.*, 2016] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[Silver *et al.*, 2017] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.